

Capture d'écran du 2025-09-27 20-47-38.png

DEVOIR MAISON

Théorie des Langages - Devoir Maison - TD3
3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersecurité
Année : 2025/2026
Enseignants : Mme.LASRI
Date : 6 mars 2026

Table des matières

1	Exercice 1 : Arbre syntaxique	2
1.1	Analyse syntaxique	2
1.2	Vocabulaire	2
2	Exercice 2 : Expressions Arithmétiques	3
2.1	Dérivation gauche de <code>id + id * id</code>	3
2.2	Arbre syntaxique correspondant	3
3	Exercice 3 : Expressions Booléennes	4
3.1	Arbre syntaxique de <code>True && False !False</code>	4
4	Exercice 4 : Grammaire Ambiguë (If-Then-Else)	5
4.1	Appartenance de la chaîne au langage	5
4.2	Ambiguïté : deux dérivations gauches différentes	5
4.3	Grammaire non ambiguë équivalente	6

1 Exercice 1 : Arbre syntaxique

Soit la grammaire context-free G définie par les règles de production suivantes :

$S \rightarrow S(S)S \mid a$

1.1 Analyse syntaxique

Donner l'arbre syntaxique (parse tree) permettant de générer la chaîne suivante :

$a(a)a$

La règle utilisée est $S \rightarrow S(S)S$, où chaque S se dérive en a via $S \rightarrow a$.

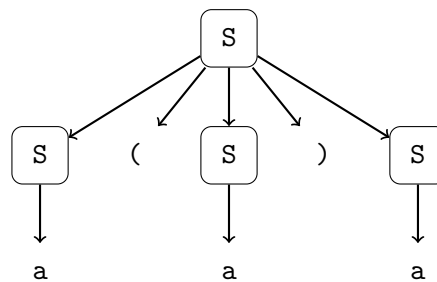


FIGURE 1 – Arbre d'analyse syntaxique de $a(a)a$

Dérivation correspondante :

$$S \Rightarrow S(S)S \Rightarrow a(S)S \Rightarrow a(a)S \Rightarrow a(a)a$$

1.2 Vocabulaire

Les symboles de la grammaire G sont identifiés comme suit :

- **Symboles Non-Terminaux (N) :** Ce sont les symboles apparaissant en partie gauche des règles de production, c'est-à-dire les symboles *substituables* :

$$N = \{S\}$$

- **Symboles Terminaux (T) :** Ce sont les symboles atomiques du langage, qui n'apparaissent jamais en partie gauche d'une règle :

$$T = \{a, (,)\}$$

Ainsi, le quadruplet définissant la grammaire G est :

$$G = (N, T, P, S) = (\{S\}, \{a, (,)\}, P, S)$$

où P désigne l'ensemble des règles de production : $S \rightarrow S(S)S \mid a$.

2 Exercice 2 : Expressions Arithmétiques

On considère la grammaire classique des expressions arithmétiques qui respecte les priorités opératoires :

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow \text{id} \mid (E) \mid -F$

2.1 Dérivation gauche de $\text{id} + \text{id} * \text{id}$

Dans une **dérivation gauche** (*leftmost derivation*), on remplace toujours le non-terminal le plus à gauche dans la forme sentencielle courante.

La dérivation complète est :

$E \Rightarrow E + T$	(règle $E \rightarrow E + T$)
$\Rightarrow T + T$	(règle $E \rightarrow T$)
$\Rightarrow F + T$	(règle $T \rightarrow F$)
$\Rightarrow \text{id} + T$	(règle $F \rightarrow \text{id}$)
$\Rightarrow \text{id} + T * F$	(règle $T \rightarrow T * F$)
$\Rightarrow \text{id} + F * F$	(règle $T \rightarrow F$)
$\Rightarrow \text{id} + \text{id} * F$	(règle $F \rightarrow \text{id}$)
$\Rightarrow \text{id} + \text{id} * \text{id}$	(règle $F \rightarrow \text{id}$)

2.2 Arbre syntaxique correspondant

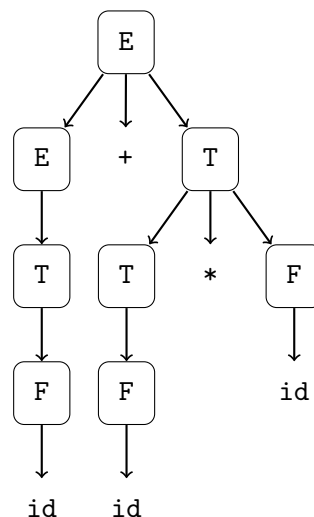


FIGURE 2 – Arbre syntaxique de $\text{id} + \text{id} * \text{id}$ — la priorité de $*$ sur $+$ est bien respectée

Remarque : La structure de l'arbre reflète les priorités opératoires : la multiplication est plus profonde dans l'arbre (sous T), ce qui signifie qu'elle est évaluée en premier, avant l'addition (sous E).

3 Exercice 3 : Expressions Booléennes

Soit la grammaire suivante modélisant des expressions booléennes :

```

E -> E || T | T
T -> T && F | F
F -> !F | (E) | True | False

```

3.1 Arbre syntaxique de `True && False || !False`

La grammaire encode les priorités suivantes (du plus fort au plus faible) :

1. `!` (NOT) — priorité la plus haute, géré par F
2. `&&` (AND) — priorité intermédiaire, géré par T
3. `||` (OR) — priorité la plus basse, géré par E

La chaîne `True && False || !False` se parenthèse donc comme :

`(True && False) || (!False)`

Dérivation gauche :

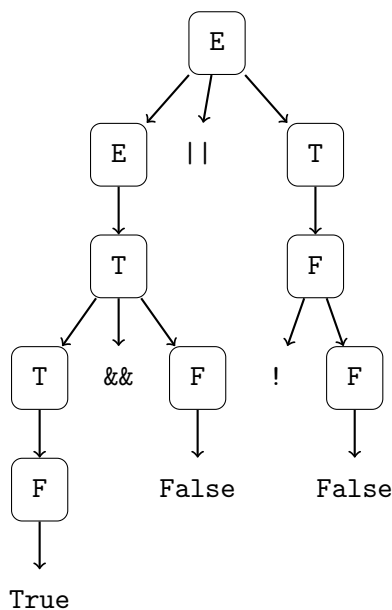
$$\begin{aligned}
 E &\Rightarrow E \parallel T \\
 &\Rightarrow T \parallel T \\
 &\Rightarrow T \&\& F \parallel T \\
 &\Rightarrow F \&\& F \parallel T \\
 &\Rightarrow \text{True} \&\& F \parallel T \\
 &\Rightarrow \text{True} \&\& \text{False} \parallel T \\
 &\Rightarrow \text{True} \&\& \text{False} \parallel F \\
 &\Rightarrow \text{True} \&\& \text{False} \parallel !F \\
 &\Rightarrow \text{True} \&\& \text{False} \parallel !\text{False}
 \end{aligned}$$


FIGURE 3 – Arbre syntaxique de `True && False || !False`

Remarque : L'arbre montre que $\&\&$ est évalué avant $\|\$, conformément aux priorités encodées dans la grammaire. L'opérateur $!$ (NOT) est le plus prioritaire, représenté au niveau le plus profond via $F \rightarrow !F$.

4 Exercice 4 : Grammaire Ambiguë (If-Then-Else)

On considère la grammaire simplifiée d'un langage de programmation contenant des structures conditionnelles :

```
S -> if E then S
S -> if E then S else S
S -> id
E -> id
```

4.1 Appartenance de la chaîne au langage

On veut montrer que la chaîne suivante appartient au langage généré par G :

if id then if id then id else id

Dérivation :

$$\begin{aligned}
 S &\Rightarrow \text{if } E \text{ then } S \text{ else } S \\
 &\Rightarrow \text{if id then } S \text{ else } S \\
 &\Rightarrow \text{if id then if } E \text{ then } S \text{ else } S \\
 &\Rightarrow \text{if id then if id then } S \text{ else } S \\
 &\Rightarrow \text{if id then if id then id else } S \\
 &\Rightarrow \text{if id then if id then id else id}
 \end{aligned}$$

La chaîne est bien générée par la grammaire. $\square$

4.2 Ambiguïté : deux dérivations gauches différentes

La grammaire est **ambiguë** car il existe **deux arbres syntaxiques distincts** pour la même chaîne, correspondant aux deux associations possibles du **else**.

Dérivation 1 — le else est associé au *if* intérieur :

$$\begin{aligned}
 S &\Rightarrow \text{if } E \text{ then } S \\
 &\Rightarrow \text{if id then } S \\
 &\Rightarrow \text{if id then if } E \text{ then } S \text{ else } S \\
 &\Rightarrow \text{if id then if id then } S \text{ else } S \\
 &\Rightarrow \text{if id then if id then id else } S \\
 &\Rightarrow \text{if id then if id then id else id}
 \end{aligned}$$

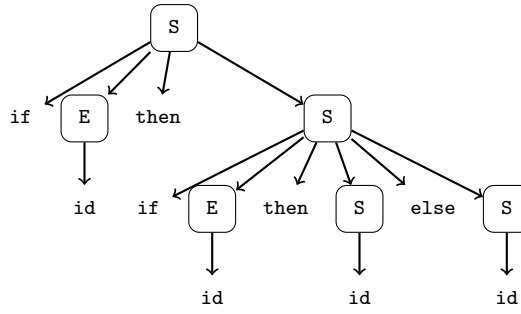


FIGURE 4 – Dérivation 1 : le **else** est rattaché au **if** intérieur (*dangling else*)

Dérivation 2 — le **else** est associé au **if** extérieur :

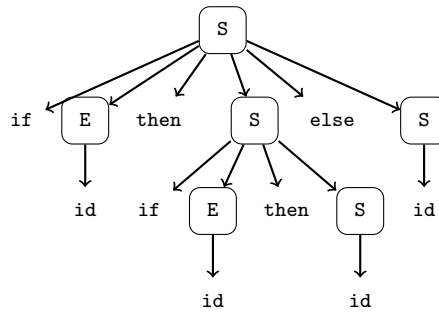
$$\begin{aligned}
 S &\Rightarrow \text{if } E \text{ then } S \text{ else } S \\
 &\Rightarrow \text{if id then } S \text{ else } S \\
 &\Rightarrow \text{if id then if } E \text{ then } S \text{ else } S \\
 &\Rightarrow \text{if id then if id then } S \text{ else } S \\
 &\Rightarrow \text{if id then if id then id else } S \\
 &\Rightarrow \text{if id then if id then id else id}
 \end{aligned}$$


FIGURE 5 – Dérivation 2 : le **else** est rattaché au **if** extérieur

Les deux dérivations gauches produisent des arbres syntaxiques **structurellement différents** pour la même chaîne, ce qui démontre que la grammaire est **ambiguë**. $\square$

4.3 Grammaire non ambiguë équivalente

La solution classique est de distinguer deux catégories d'instructions : les **instructions complètes** (*matched*, notées *M*) et les **instructions incomplètes** (*unmatched*, notées *U*). La convention adoptée est : **chaque else est toujours associé au if le plus proche** (*nearest unmatched if*).

S $\rightarrow$ **M** | **U**

M $\rightarrow$ **if** **E** **then** **M** **else** **M**

M $\rightarrow$ **id**

U	->	if E then S
U	->	if E then M else U
E	->	id

Explication :

- M (matched) : représente une instruction où chaque **if** a bien son **else** correspondant.
- U (unmatched) : représente une instruction où au moins un **if** n'a pas encore de **else**.
- La règle $U \rightarrow \text{if } E \text{ then } M \text{ else } U$ impose que la branche **then** d'un **if-then-else** soit toujours une instruction *matched*, ce qui force le **else** à se rattacher au **if** le plus proche et élimine l'ambiguïté.

Vérification sur la chaîne `if id then if id then id else id` :

$$\begin{aligned}
 S &\Rightarrow U \\
 &\Rightarrow \text{if } E \text{ then } S \\
 &\Rightarrow \text{if id then } S \\
 &\Rightarrow \text{if id then } M \\
 &\Rightarrow \text{if id then if } E \text{ then } M \text{ else } M \\
 &\Rightarrow \text{if id then if id then } M \text{ else } M \\
 &\Rightarrow \text{if id then if id then id else } M \\
 &\Rightarrow \text{if id then if id then id else id}
 \end{aligned}$$

Cette nouvelle grammaire ne possède désormais qu'un seul arbre syntaxique pour cette chaîne, garantissant ainsi son **non-ambiguïté**.

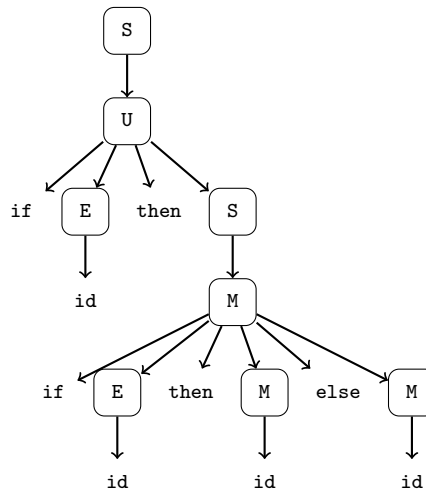


FIGURE 6 – Arbre syntaxique de la grammaire non ambiguë — le **else** est forcément rattaché au **if** intérieur